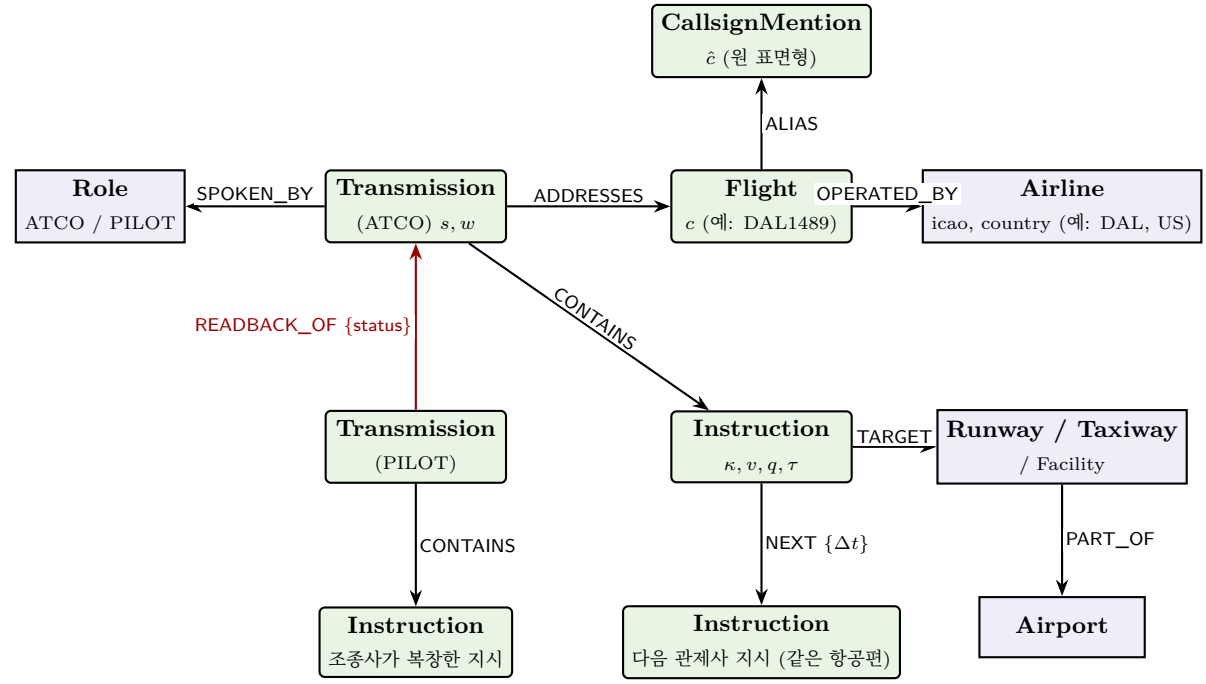
KG 스키마 그림 후보 비교

여섯 후보 모두 **같은 KG** 를 그린 것이다 (노드 종류 Transmission·Instruction 은 각각 하나). 다른 점은 그리는 방식과, D 만 복창 간선을 지시 단위로 바꾸는 구조 변경이라는 점이다.

후보	그리는 방식	장점	단점
A (현재)	예시식: Transmission·Instruction 을 여러 번 그림	관계가 구체적으로 보임	노드 종류가 나뉜 것처럼 보임
B	스키마식: 종류마다 한 번, 같은 종류끼리 관계는 자기 화살표 (self-loop)	노드 종류와 관계가 한눈에 보임	시간 흐름 (, 순서) 은 안 보임
C	B + KATL 실제 예시 시간축	구조 (B) 와 시간·복창 (예시) 을 함께 보여줌	그림이 2 개
D	B 와 같되 READBACK_OF 를 지시끼리 연결 (구조 변경)	어느 지시가 복창됐는지 그래프에서 바로 보임	코드 수정 필요, 간선 수 증가
E	B 와 같되 시각을 가진 Transmission·Instruction 을 “시간축 위의 사건” 상자로 묶고 시간 화살표 표시	무엇이 시간에 따라 쌓이는지 한눈에 보임	(C 처럼 예시 시간축을 함께 두는 것이 좋음)
F	E 를 세 층 (사건 / 개체 / 정적 KB) 으로 정리 + 시간축 예시. 선행 연구 (Zep) 구조를 따름	층마다 성격이 분명하고 선행 연구로 설명 가능	그림이 큼

후보 A: 현재 그림 (예시식)

관제사 송신과 조종사 송신, 지시들을 **각각 따로** 그려 관계를 보여준다. 실제 KG 에서는 Transmission·Instruction 이 각각 한 종류이고, 관제사/조종사는 속성 `speaker` 로 구분된다.

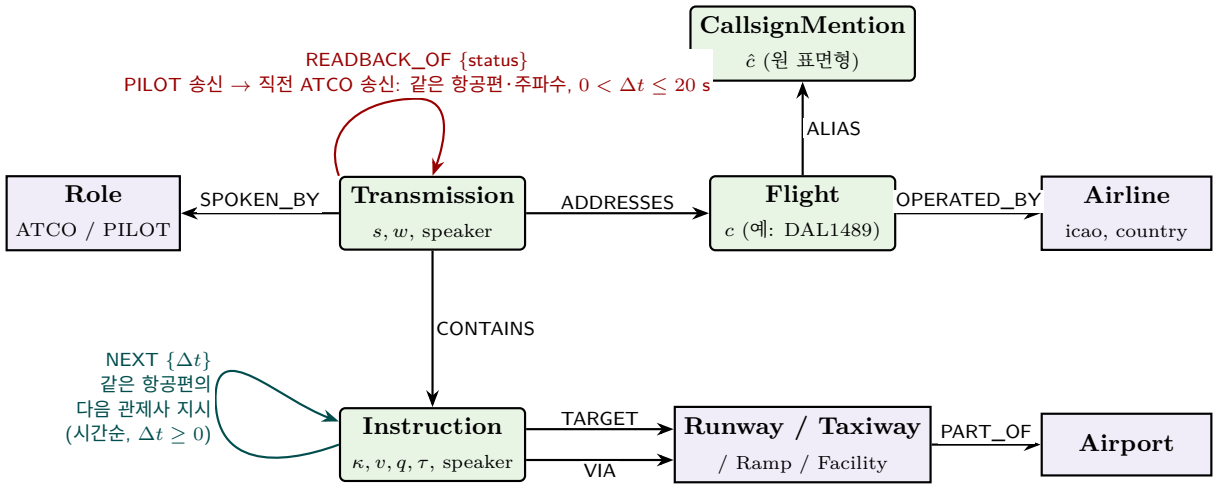


동적 (사건) 계층: 유효 구간 τ, 근거 송신 prov 보유
모든 간선: 적재 시각 t' (`ingested_at`)
정적 계층 $\mathcal{K}_S$ (atmonto IRI 매핑)

후보 B: 스키마식 (종류마다 한 번 + 자기 화살표)

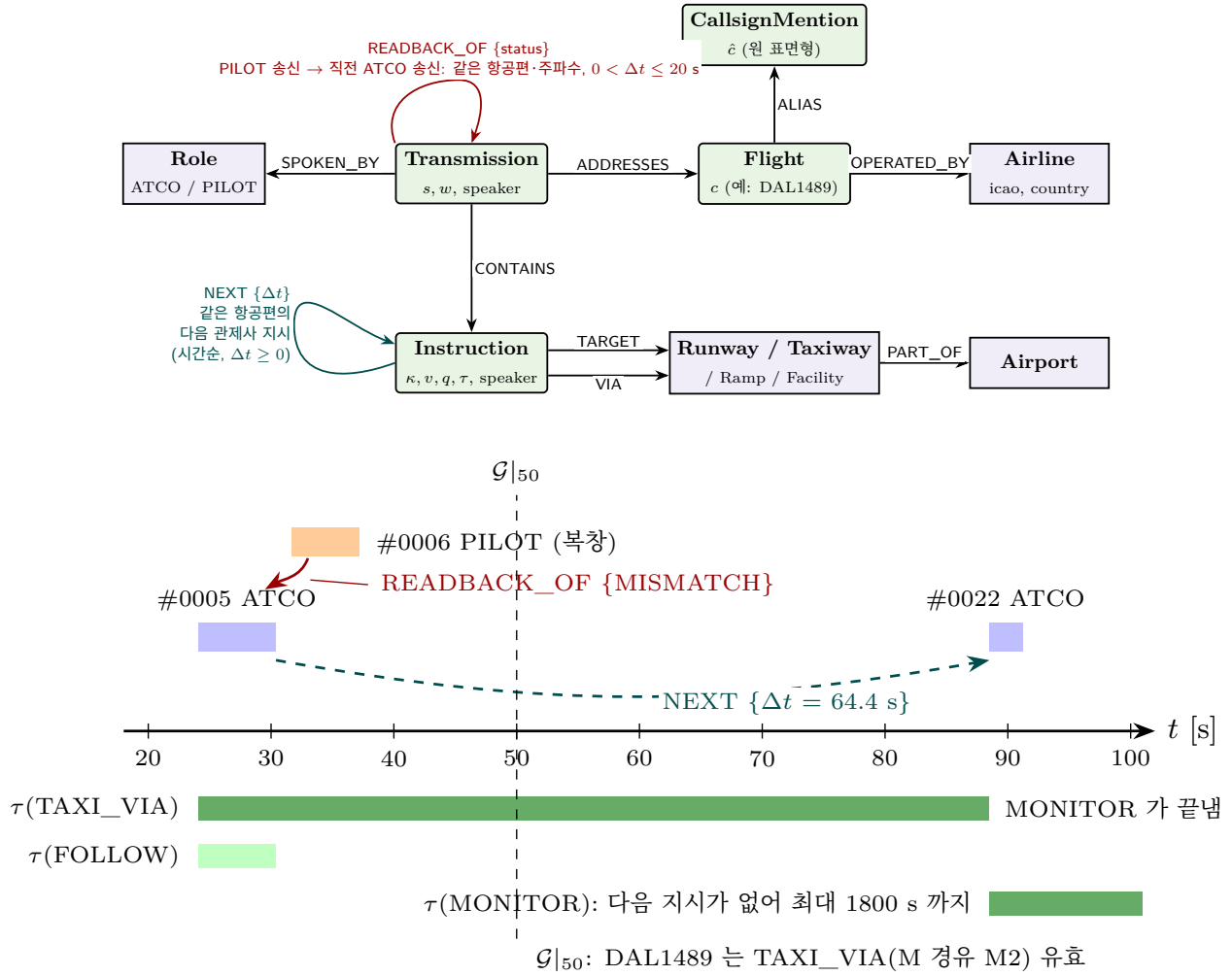
Transmission·Instruction 을 **한 번씩만** 그리고, 같은 종류끼리의 관계는 자기 자신으로 돌아오는 화살표로 그린다. 관제사/조종사는 상자 안 `speaker` 속성으로 표시한다.

- READBACK_OF: 조종사 송신 → 관제사 송신 (복창 관계)
- NEXT: 같은 항공편의 관제사 지시 → 다음 관제사 지시 (시간 순서)



후보 C: 스키마 (B) + 실제 예시 시간축 (추천)

위는 후보 B 의 스키마 (어떤 노드와 관계가 있나), 아래는 KATL 지상관제 DAL1489 의 실제 송신으로 그린 시간축 (실제로 어떻게 연결되나) 이다. 아래 그림이 기존 SFO 시간 모델 그림을 대신한다.



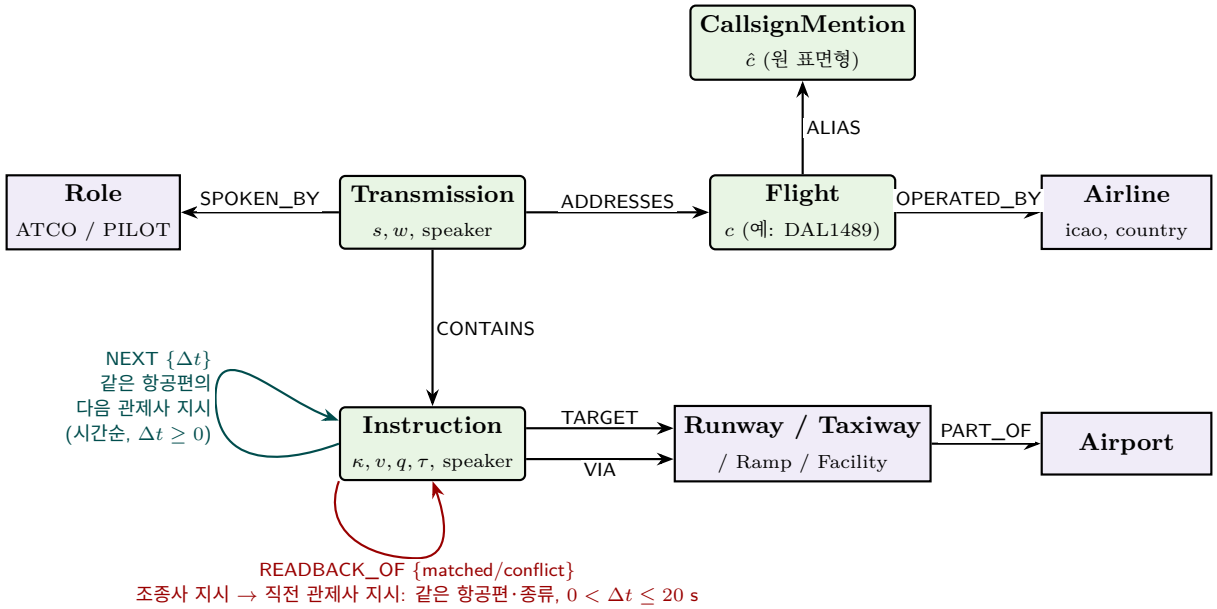
KATL 전체 (Whisper) 에서 연결된 복창 404 건 중 351 건이 관제사 송신 끝난 뒤 5 s 안, 95% 가 12.4 s 안에 시작했다. 아래 예의 #0006 도 #0005 끝난 뒤 1.3 s 만에 시작한다.

#0005	24.1 s, ATCO	“Delta fourteen eighty nine pass behind … nine left to Mike two taxi via Mike” → FOLLOW, TAXI_VIA (M 경유 M2), TRAFFIC_INFO
#0006	31.7 s, PILOT	“Nine left Mike two via Mike … Delta fourteen eighty nine” → TAXI_VIA, FOLLOW (복창)
#0022	88.5 s, ATCO	“Delta fourteen eighty nine monitor twenty three eighty five” → MONITOR

파란 상자 = 관제사 송신, 주황 상자 = 조종사 송신, 초록 막대 = 지시의 유효 구간 τ. 시각은 2024-08-26 00:00:00Z 부터의 초. NEXT 는 실제로 #0005 의 마지막 지시 (TRAFFIC_INFO) 에서 #0022 MONITOR 로 이어진다.

후보 D: 복창을 지시 단위로 연결 (구조 변경)

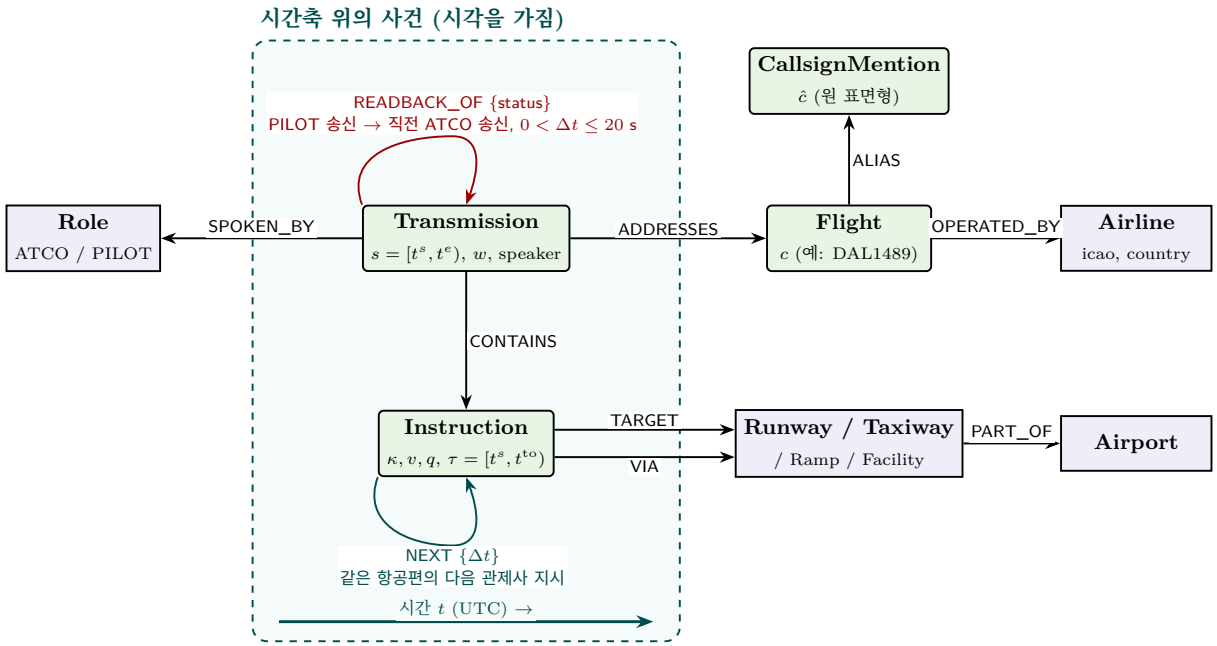
READBACK_OF 를 송신끼리가 아니라 **조종사 지시 → 관제사 지시**로 잇는다. 지금은 어느 지시가 맞았는지가 송신 간선의 속성 (`matched`, `conflicts`) 에 들어 있는데, 이를 간선 자체로 만든다. 예: #0006 의 TAXI_VIA → #0005 의 TAXI_VIA {conflict: target}. **코드 수정 (M4, M5) 이 필요하다.**



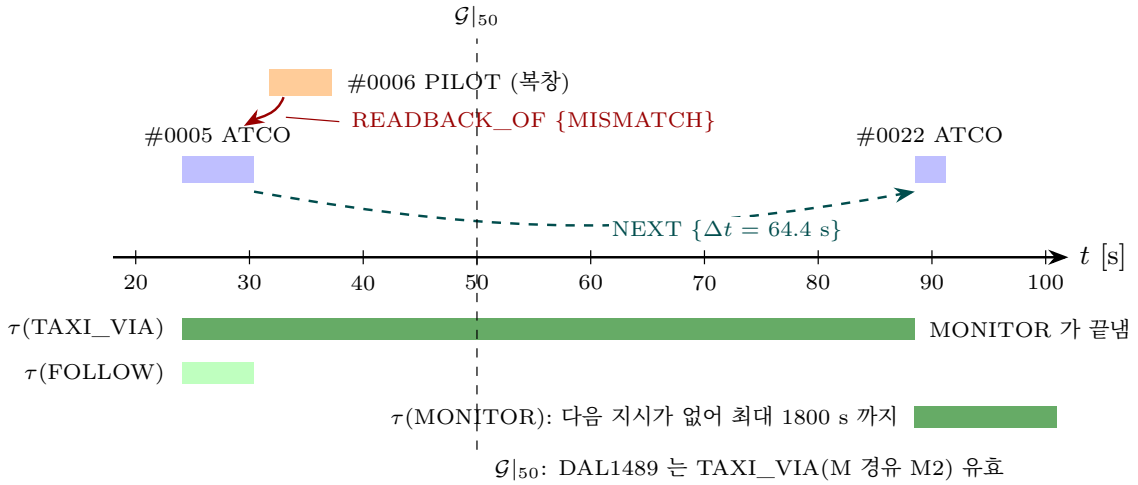
후보 E: 시각을 가진 노드를 한 묶음으로 (C 의 스키마를 대체)

시각을 가진 노드는 **Transmission 과 Instruction 둘뿐**이다. 두 노드를 점선 상자 "시간축 위의 사건" 으로 묶고, 상자 아래에 시간 화살표를 둔다. 묶음 안의 관계 (CONTAINS, READBACK_OF, NEXT) 는 모두 시간축 위에서 일어난다.

- 지시의 유효 구간 τ 는 그 지시를 담은 송신의 시작 시각 t^s 에서 시작한다 (같은 시간축).
- 묶음 밖의 Flight, CallsignMention 은 송신에서 생기지만 시각을 갖지 않는다. 언제 교신했는지는 연결된 송신으로 안다.
- Role, Airline, 활주로·유도로, Airport 는 정적 지식이다.
- 노드를 합치지 않고 그림에서만 묶는다. 송신 하나에 지시가 여러 개 (DLR: Callsign + Command 여러 개) 인 구조를 유지하기 위해서다.



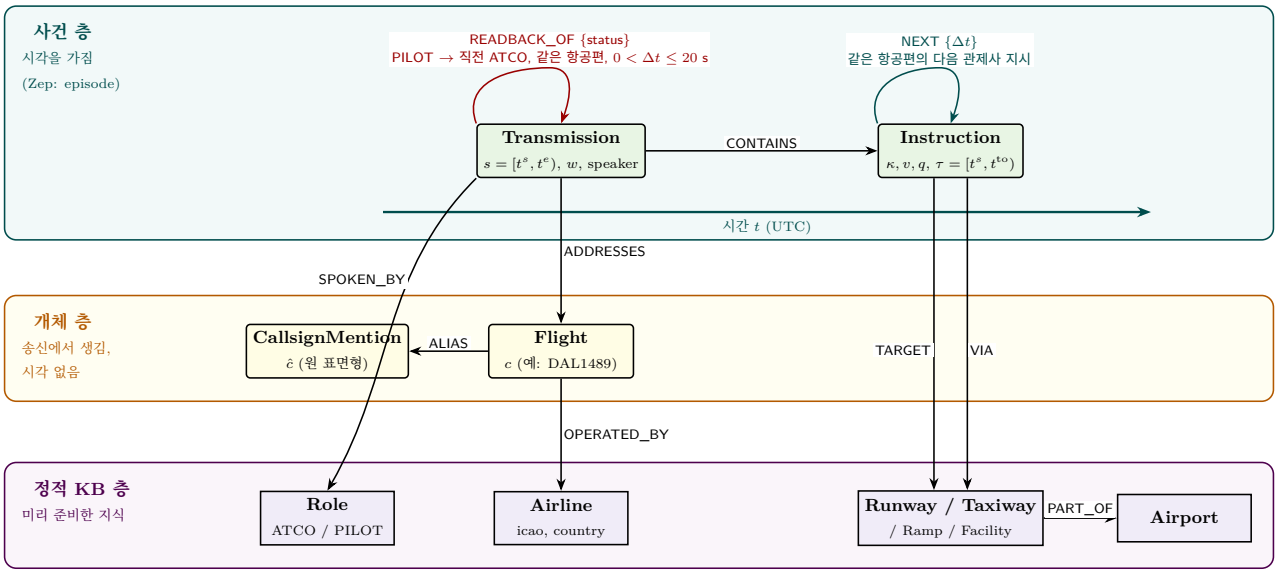
아래는 후보 C 와 같은 KATL 시간축 예시다. 묶음 상자 안의 관계가 실제로 시간 위에 어떻게 놓이는지 보인다.



후보 F: 세 층 구조 (Zep 식) + 시간축 예시

노드를 **성격에 따라 세 층**으로 나눈다. 선행 연구 Zep (Rasmussen et al., arXiv 2501.13956, 2025)(대화 메시지 → temporal KG) 의 에피소드 층·개체 층 구분을 따랐다.

층	노드	성격
사건 층	Transmission, Instruction	무전에서 생기고 **시각을 가짐**. 시간축 위에 쌓인다. Zep 의 episode(원본 메시지) 에 해당
개체 층	Flight, CallsignMention	무전에서 생기지만 **시각이 없음**. 언제 교신했는지는 연결된 송신으로 안다. Zep 의 entity 에 해당
정적 KB 층	Role, Airline, Runway/Taxiway/Ramp/Facility, Airport	미리 준비한 지식 (katl.json, airlines.json). 변하지 않음



간선의 근거: READBACK_OF 는 대화 구조 연구의 reply-to(발화 → 응답한 발화) 를 ATC 에 맞게 좁힌 것 (다른 화자, 20 s 안, 같은 항공편). NEXT 는 사건 그래프의 next-event 관계로, 지시의 시작 시각 순서에서 유도된다. 이전 지시를 끝낼 때는 지우지 않고 τ 의 끝만 정한다 (Zep 의 invalidation 과 같은 방식). 모든 간선에는 적재 시각 t'(`ingested_at`) 가 붙는다.

